OPEN SOURCE TOOLS FOR DATA SCIENCE

Project Title

LicenseLens — Open-Source Dependency License Compliance Auditor

Programme	B.Sc. (Artificial Intelligence)
Academic Year	2026-27
Team ID	OST-19
Team Members	Sanobar Shaikh – 25030421039 Prajakta Suryavanshi - 25030421029
Faculty	Dr. Nabanita Choudhury, Assistant Professor

SYMBIOSIS ARTIFICIAL INTELLIGENCE INSTITUTE (SAII)
Constituent of Symbiosis International (Deemed University)
Lavale, Pune - 412115

1. Abstract

Most modern software projects rely on open-source libraries instead of writing every part of the code from scratch. A typical Python project may use many libraries, and each library comes with its own license. Issues can arise when these licenses are not compatible. For example, using a strongly copyleft license such as GPL-3.0 may create an obligation to release source code under the same license in certain circumstances. For students and small teams, checking every dependency manually can take time and mistakes are easy to make.

We developed LicenseLens as a simple way to make this process easier. The user provides a project's requirements.txt file, and the tool checks each dependency's declared license using the public PyPI registry. It then groups the results into low, medium, high, or conflict categories and displays them on a dashboard. Users can also download a Software Bill of Materials (SBOM) and a compliance report. LicenseLens runs through three Docker containers: a Streamlit dashboard, a FastAPI backend containing the scanning logic, and a PostgreSQL database. Docker Compose connects these services, allowing us to demonstrate containerisation and multi-service orchestration in a practical project.

We tested LicenseLens using our own project's dependencies, which showed no conflicts, and a separate test list containing GPL-licensed packages, which the tool flagged as conflicts. Overall, the project brings together open-source licensing, Git/GitHub collaboration, and Docker-based deployment—the main areas we were expected to demonstrate in this course.

2. Introduction

Open-source software makes code available for others to use and build on, but each project still has a license that sets out the conditions for use. Permissive licenses such as MIT, BSD, and Apache-2.0 generally have fewer restrictions. Copyleft licenses such as GPL can require derivative works to follow the same licensing terms. If developers combine dependencies without checking their licenses, they may unintentionally take on obligations or create compliance issues.

The risk is not always obvious from a requirements.txt file. Even if it contains dozens of packages, it does not clearly show which ones use GPL, LGPL, or an unknown license. Checking every package's PyPI page individually can be time-consuming, so it is easy for this step to be missed. Tools such as FOSSA and Snyk, along with Software Bills of Materials (SBOMs), help teams keep track of software components and their licenses. SBOMs are increasingly useful for reviews and audits.

For this course, we wanted to build something that used open-source tools as part of the actual solution, rather than adding Git and Docker only to meet the requirements. License compliance was a good fit because it is a real issue in software development and the application could be divided into services that work together through Docker Compose. It also gave us a direct way to explore open-source licensing, a topic included in our syllabus.

3. Problem Statement and Objectives

Problem Statement: Open-source projects often use third-party packages without checking whether their licenses are compatible. This can lead to legal or compliance concerns. For example, including a GPL-licensed library in an MIT-licensed project may create obligations to release the project under GPL, depending on how the library is used. Small teams and student developers need a straightforward way to identify possible issues before releasing their work.

Objectives:

- Automatically detect the license of every package listed in a project's requirements.txt by querying the PyPI registry.
- Classify each license by risk level (low, medium, high, or conflict) against the project's own declared license.
- Present results on a clear, usable dashboard rather than a raw JSON dump.
- Generate a standards-based SBOM (CycloneDX format) and a downloadable compliance report.
- Package the application as separate, communicating Docker containers, orchestrated with Docker Compose, so the whole system starts with a single command on any machine.

4. Dataset / Data Source

LicenseLens does not rely on a fixed dataset such as a pre-made CSV file. Instead, it retrieves package license information from a public registry whenever a scan is run. This approach is more practical for a tool that needs to work with packages whose information may change over time.

- Dataset name: PyPI Package License Metadata.
- Source: Python Package Index (PyPI) public JSON API, at pypi.org/pypi/<package-name>/json — no authentication required.
- Fields used: package name, installed version, the license classifier/string PyPI reports for that package.
- License reference standard: the SPDX License List (spdx.org/licenses), which we use to normalise license strings (e.g. recognising "GPLv3" and "GPL-3.0" as the same license) before applying our risk rules.
- Format and type: JSON responses from the PyPI API, parsed into a tabular structure (one row per dependency) for display and export.

Since the tool fetches license information directly from PyPI, it can reflect updated package metadata when available. It can also be used with different Python projects rather than being limited to a single prepared dataset.

5. Open-Source Technology Stack

The project uses open-source tools throughout. The table below lists each technology, its version, and the role it plays in LicenseLens.

Tool / Technology	Version	Purpose in the Project
Python	3.11	Main language used for the scanner, backend, and dashboard logic.
FastAPI	0.115.x	Provides the backend REST API, including the /scan endpoint and license-scanning module.
Streamlit	1.38.x	Provides the dashboard for uploading files, viewing results and risk charts, and downloading reports.
Pandas	2.2.x	Organises scan results into tables and supports SBOM and report generation.
Uvicorn	0.30.x	Runs the FastAPI backend as an ASGI server.
PostgreSQL	15	Stores scan history so previous results can be viewed and compared.
Docker & Docker Compose	current	Runs the dashboard, API, and database as three connected services with one command.
Git & GitHub	—	Tracks code changes and supports branches, pull requests, and team code review.
GitHub Actions	—	Automatically runs the test suite whenever code is pushed.

Table: Open-source technology stack used in LicenseLens.

6. System Architecture / Workflow

LicenseLens processes an uploaded dependency file through a series of steps and produces a compliance report. These steps are handled by three Docker containers that communicate over a shared Docker Compose network.

Step	Component	What happens
1	Upload	The user uploads a requirements.txt file through the Streamlit dashboard.
2	API call	The dashboard sends the file to the backend's /scan endpoint in FastAPI.
3	License lookup	The scanner requests each package's declared license from the PyPI JSON API.

4	Risk classification	The scanner applies the project's risk rules—Low (MIT/BSD/Apache), Medium (LGPL/MPL/EPL), and High (GPL/AGPL)—and flags possible conflicts with the project's own license.
5	Storage & output	Results are saved in PostgreSQL and sent back to the dashboard, where users can view the table and risk chart and download the SBOM or report.

Table: End-to-end scan workflow, from upload to report.

The Streamlit dashboard does not connect directly to PyPI or PostgreSQL. Instead, it sends requests to the FastAPI backend, which handles communication with both the public PyPI API and the database. This separation helped us divide the work: one team member focused on the backend and scanning logic, while the other worked on the dashboard. We agreed on the API response format first, then connected the two parts.

Risk classification rules applied by the scanner:

- The scanner uses the following rules to group license risk:
- Medium risk: LGPL, MPL, EPL — weak copyleft, usually fine but worth reviewing.
- High risk: GPL, AGPL — strong copyleft, can force source disclosure.
- Conflict: any high-risk license detected inside a project that is itself declared under a permissive (non-copyleft) license.

7. Implementation

We divided the implementation according to the main parts of the system. One team member worked on the FastAPI backend and the license-scanning and risk-classification logic, while the other developed the Streamlit dashboard and the SBOM/report exports. We agreed on the JSON response format early, so the dashboard could be developed and tested with sample data while the scanner was still in progress. Once the backend was ready, we connected it without needing to rebuild the interface.

Data loading and preprocessing:

- The uploaded requirements.txt file is read line by line. Blank lines and comments are ignored, and the package names and version information are extracted.
- For each package, the scanner requests its metadata from the PyPI JSON endpoint. Pandas is then used to organise the results into one dataframe containing the project's dependencies.

Scanning / risk engine:

- The license strings returned by PyPI are matched to SPDX identifiers, helping the tool handle different ways of writing the same license, such as “GPLv3” and “GPL-3.0.”
- After normalising each license, the scanner assigns a risk category and checks whether it conflicts with the license declared for the project.

Dashboard / interface:

- The Streamlit dashboard includes a file upload option, a repository URL field, and a “Run Scan” action. Results appear in a colour-coded table with a risk-distribution chart. An expandable “What needs attention” section highlights flagged packages and suggests what to review.
- Users can export the scan results as a CycloneDX-format SBOM in JSON and as a plain-text compliance summary. Both are generated from the same scan dataframe when requested.

The Results and Output section includes spaces for screenshots of the dashboard summary, license breakdown, and flagged issues.

8. GitHub Repository and Version Control

GitHub Repository Link:

prajaktasuryavanshi/licenselens:

We used feature branches instead of making all changes directly on main. Each feature was developed separately and merged through a pull request after review by the other team member.

- We used the branches feature/scanner-backend, feature/dashboard-ui, and feature/sbom-export, while keeping main as the stable branch.
- Our commit messages used prefixes such as feat:, fix:, docs:, and chore: to make it easier to understand what each change was for.
- Example commit messages include: feat: add FastAPI /scan endpoint; feat: add PyPI license lookup; and fix: handle long license strings.
- The pull requests covered the scanner module (#1), the backend and dashboard—including the /scan endpoint, risk engine, SBOM and CSV export (#2), and caching, tests and CI (#3).
- We also used pull-request comments to discuss changes before merging. A shared .gitignore kept files such as the virtual environment and .env file out of version control from the beginning.

9. Reproducibility and Environment Setup

We set up the project so that someone reviewing it can clone the repository and start the full application with one command, without installing its Python packages directly on their machine.

- Software requirements: Docker and Docker Compose installed on the host machine (no local Python installation needed to run the full system).
- The backend and dashboard each have their own requirements.txt file. The backend uses FastAPI, Uvicorn, Pandas and Requests, while the dashboard uses Streamlit, Pandas and Requests. Each set of dependencies is installed inside its own container.

- To start the project: (1) clone the repository with git clone, (2) create a .env file using the provided .env.example and add the database credentials, and (3) run docker compose up --build from the project root.
- Once running: the dashboard is available at localhost:8501, the backend API and its interactive docs at localhost:8000/docs.
- Database settings—including the host, username and password—are read from environment variables in .env and passed to the containers through docker-compose.yml. This avoids hard-coding credentials in the source code.

10. Containerization / Deployment

Containerisation is an important part of the assignment and of LicenseLens itself. The application runs as three services on one Docker Compose network: the dashboard, the API (which contains the scanner), and the database.

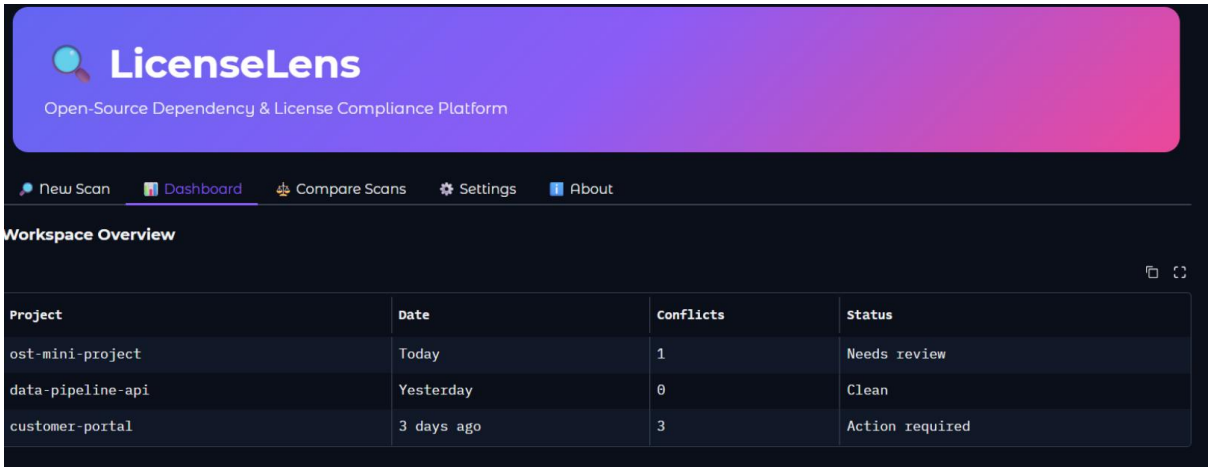
- The dashboard and API each have their own Dockerfile. Both are built from a lightweight Python base image and install only the dependencies required by that service.
- The database service uses the official postgres:15 image and a named volume to preserve scan history when containers are restarted.
- Running docker compose up --build builds the dashboard and API images and starts all three services. The API service is configured to wait for the database using a depends_on condition.
- All three services share an internal Compose network and communicate using service names. For example, the dashboard connects to http://api:8000, so fixed container IP addresses are not needed.
- For this project, the application is intended to run on any machine with Docker installed, such as a reviewer's laptop or a classroom demonstration computer. Cloud hosting was not necessary for our project timeline and reproducibility goals.

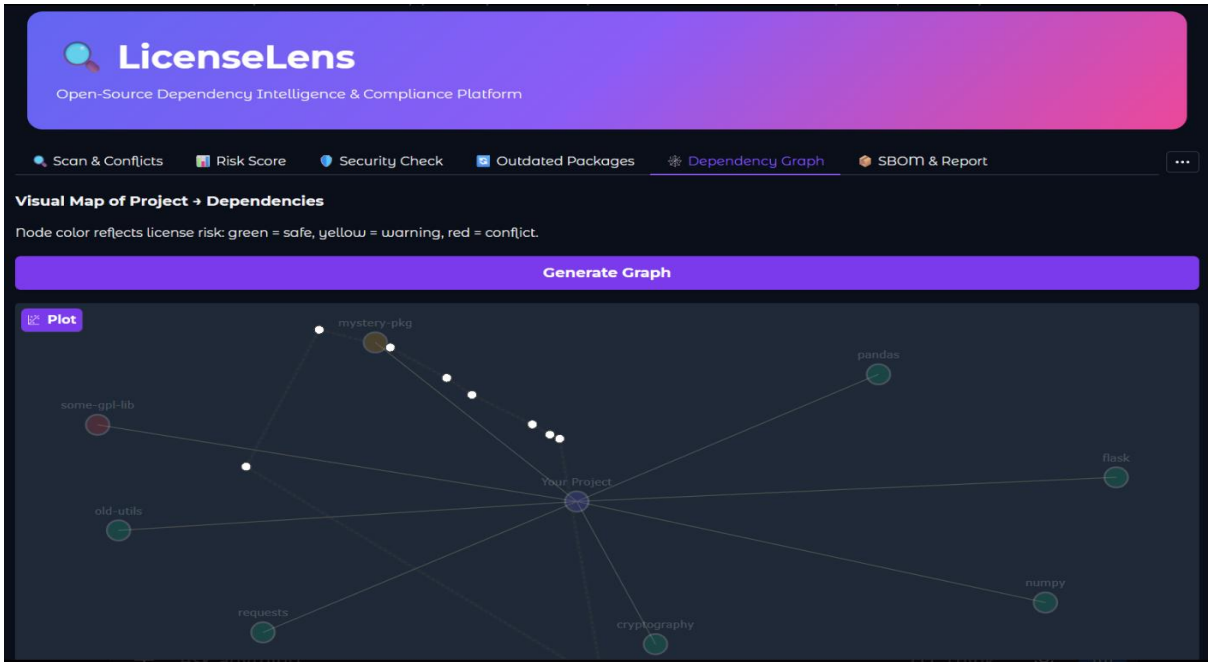
11. Results and Output

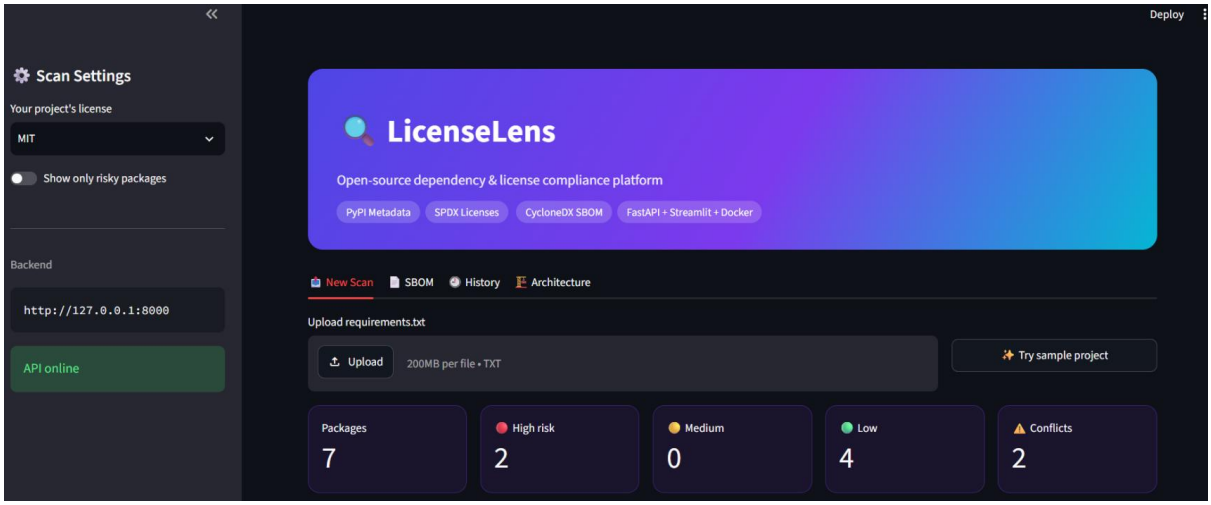
We tested LicenseLens with two different dependency lists to check how it behaved in more than one situation.

- Our own project's dependencies: the five packages we tested—FastAPI, Streamlit, Pandas, Uvicorn and Requests—were reported with permissive licenses such as MIT, BSD or Apache-2.0. The scan showed no conflicts, which matched our expectations because we had avoided GPL packages in our own dependency list.
- Risky dependency list: we added pylint, PyQt5 and mysqlclient, described in our test case as GPL-licensed, to a project declared under MIT. LicenseLens flagged three conflicts. When we changed the project's declared license to GPL-3.0, the conflicts cleared. This test

showed that the tool compares package licenses with the project's declared license rather than flagging every GPL license automatically.







The results suggest that LicenseLens can retrieve license details from a public source, apply the project's defined risk rules consistently, and display the findings in a way that is easier for a non-technical reviewer to understand.

12. Open-Source Licensing and Dependencies

The main third-party packages used in the project and their licenses are listed below:

- FastAPI — MIT License
- Streamlit — Apache License 2.0
- Pandas — BSD 3-Clause License
- Uvicorn — BSD 3-Clause License
- Requests — Apache License 2.0
- PostgreSQL — PostgreSQL License (a permissive, MIT-style license)

These licenses are permissive and are compatible with releasing LicenseLens under the MIT License. We chose MIT for our project because it is a widely used, permissive option and is compatible with most of the licenses in our technology stack.

The license metadata is retrieved from PyPI's public JSON API. The project uses the SPDX License List (spdx.org/licenses) as a reference for identifying and normalising license names. SPDX is an open standard maintained by the Linux Foundation.

13. Challenges Faced and Solutions

- PyPI returns license information in different text formats. For example, the same license may appear as “GPLv3,” “GPL-3.0,” or “GNU General Public License v3.” We handled this by normalising the returned strings against the SPDX identifier list before applying the risk rules, instead of relying on exact text matches.
- Some packages do not provide clear, machine-readable license information on PyPI. Instead of ignoring these packages, LicenseLens labels them “Unknown” and displays a warning, since missing license information may also need attention.
- The dashboard and backend depended on each other for testing, which could have slowed down development. To avoid blocking either side, we agreed on the API's JSON response format early and built the dashboard using mock data in that format. When the real scanner was ready, we could connect it without changing the interface.
- At first, the API container sometimes started before PostgreSQL was ready, which caused connection errors. We addressed this by adding a database health check and a depends_on condition in docker-compose.yml.

14. Limitations

- At present, the scanner reads Python requirements.txt files only. It does not yet support package.json or dependency files from other ecosystems.
- The tool uses a fixed set of rules to assign low, medium, high, or conflict labels based on license families. It does not use a complete legal compatibility matrix, so its results should be treated as an initial check rather than a replacement for legal advice.
- The scanner checks the direct dependencies listed in requirements.txt, not every dependency they may install. As a result, it could miss a license issue in a deeper, transitive dependency.
- The dashboard includes a field for a GitHub repository URL, but automatic repository scanning—cloning the repository and reading its requirements.txt—is not fully implemented yet. Uploading the file directly is the reliable way to run a scan at present.

15. Conclusion and Future Scope

Licenselens achieves the main goal of the project: it checks a dependency file for possible license risks and conflicts without requiring every package to be reviewed manually. It is also built as a multi-service application using Docker, rather than as a single script. Working on the project gave us practical experience with GitHub branches and pull requests, communication between services in Docker Compose, and the role of open-source licenses in software development.

If we continue developing LicenseLens, some useful improvements would be:

- Add support for package.json and dependency files from other ecosystems, so the tool can be used beyond Python projects.
- Scan the full transitive dependency tree, not only the direct dependencies.
- Allow users to scan a GitHub repository automatically instead of uploading requirements.txt themselves.
- Add a scan-history page so teams can review how their license compliance results change between releases.

16. References

- Python Package Index (PyPI) JSON API — https://pypi.org/pypi/
- SPDX License List — https://spdx.org/licenses/
- CycloneDX SBOM Specification — https://cyclonedx.org/
- FastAPI Documentation — https://fastapi.tiangolo.com/
- Streamlit Documentation — https://docs.streamlit.io/
- Docker Compose Documentation — https://docs.docker.com/compose/
- GitHub Actions Documentation — https://docs.github.com/actions

17. Team Contribution

Team Member	PRN	Specific Contribution
Sanobar Shaikh	25030421039	Developed the FastAPI backend and scanner; added PyPI license lookups and risk-classification rules; integrated the PostgreSQL schema; configured Docker Compose and container networking; and set up GitHub Actions CI.
Prajakta Suryavanshi	25030421029	Developed the Streamlit dashboard, including file upload, results table, risk charts, and the “what needs attention” panel; added CycloneDX SBOM and compliance-report exports; created the dashboard Dockerfile; and refined and tested the interface with mock and live data.

Table: Individual contributions of team members.

18. Annexure

Annexure A: Sample Requirements File for Risky Dependency Testing(risky_requirements.txt)

flask==2.0.1

requests==2.31.0

numpy

pandas

django

pylint

PyQt5

Mysqlclient

Annexure B — Requirements File for LicenseLens Project

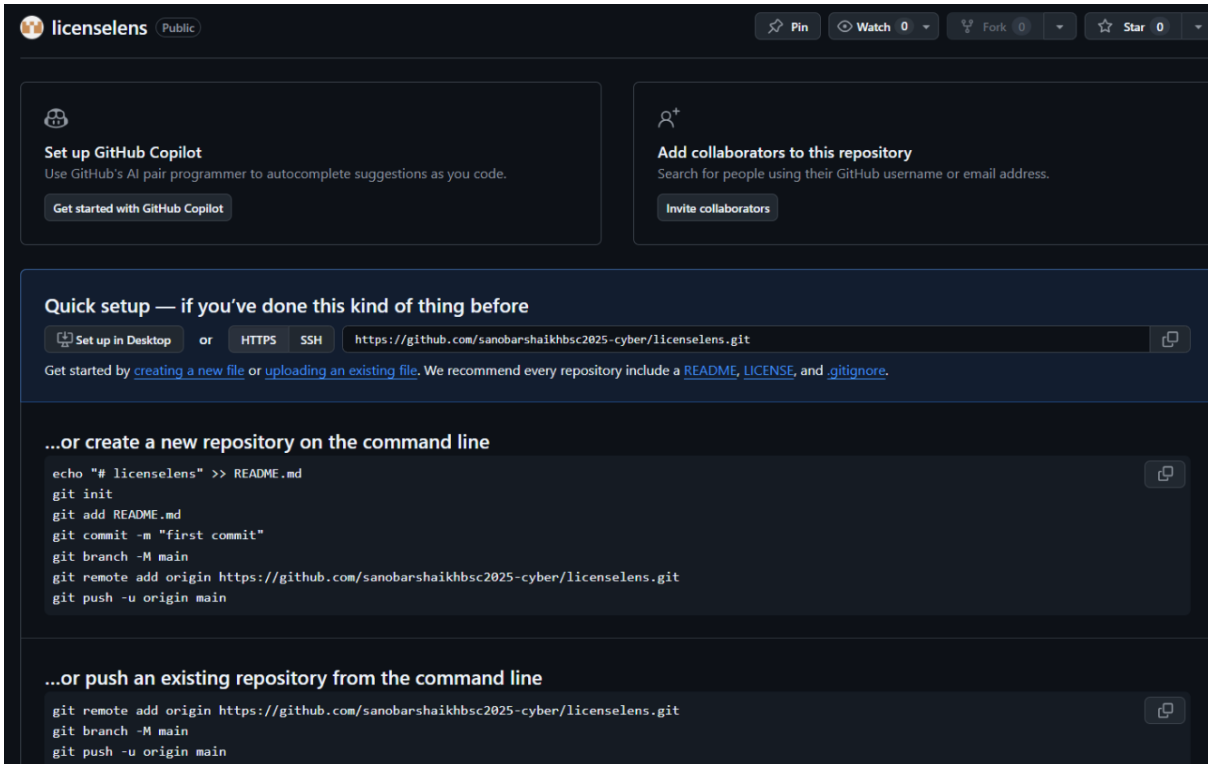
fastapi

uvicorn

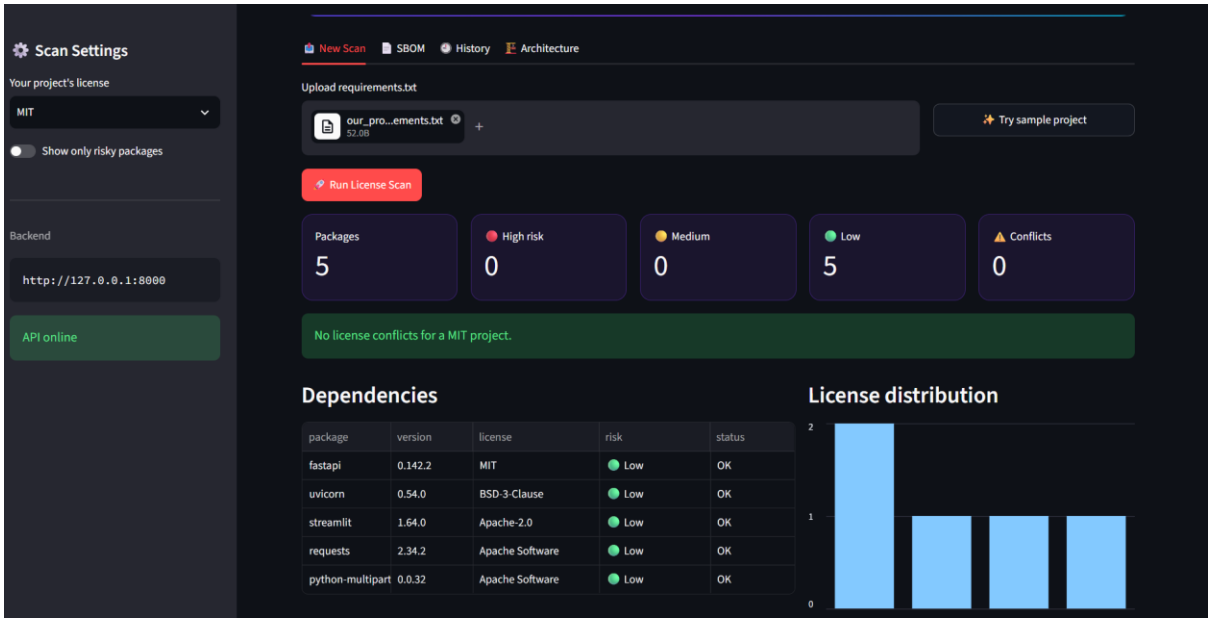
streamlit

requests

python-multipart



GitHub repository



Project Dependency Graph Visualization